Department of Software Engineering
BAHRIA UNIVERSITY KARACHI CAMPUS

Discovering Knowledge

COURSE: CEL 220
COMPUTER ARCHITECTURE & LOGIC DESIGN
PROJECT REPORT

CLASS: BSE – 3C (FALL - 2024)

Project Title
RPG GAME SIMULATION

Group Members

Student Name	Enrollment#
FIZA KHAN (Team Lead)	02-131232-048
ALISHBA MAQSOOD	02-131232-118
M.ABUBAKAR AFZAL	02-131232-052
MUSTAJAB HAIDER CHEEMA	02-131232-081

Submitted to:

Course Instructor: Engr. Ramsha Mashood
Lab Instructor: Engr. Noor us Sabah

TABLE OF CONTENTS

ABSTRACT **3**

1. INTRODUCTION **3**

2. PROBLEM STATEMENT **3**

3. PROPOSED SOLUTION **4**

1.1 3.1. FEATURES OF THE PROJECT 4

1.2 3.2. METHODOLOGY 4

1.3 3.3. TECHNOLOGIES TO BE USED 5

4. PROJECT SCOPE **5**

5. MODULE DISTRIBUTION **5**

- Fiza Khan: Game Logic & Combat System 5
- M Abubakar Afzal: Character Initialization & Inventory System 6
- Alishba Maqsood: Random Event Generation & Exploration 6
- Mustajab Haider Cheema: Boss Battle & Game End Logic 6

6. code **7**

7. Interfaces **25**

8. CONCLUSION **29**

9. REFERENCES **29**

ABSTRACT

This project focuses on the development of a text-based RPG game simulation using MIPS Assembly language, aimed at demonstrating the capabilities of assembly programming in creating engaging game mechanics. The simulation incorporates essential RPG features such as character creation, turn-based combat, leveling up, random events, and item management, all implemented directly in MIPS Assembly. The objective is to address the lack of text-based RPGs that seamlessly integrate with MIPS architecture, providing a nostalgic gaming experience while highlighting the intricacies of low-level programming. The project involves a structured approach to development, including planning key game mechanics, designing data structures for character stats and items, and implementing a main game loop that facilitates gameplay. Ultimately, the final product serves as both an educational tool for understanding assembly language and a functional RPG that emphasizes memory management and system calls.

1. INTRODUCTION

This project aims to create an **RPG game simulation** using **MIPS Assembly**, so that it demonstrates the power of assembly language in simulating game mechanics. It will involve key RPG features such as character creation, combat, leveling up, random events, and item management-all directly implemented in MIPS Assembly.

2. PROBLEM STATEMENT

It is the purpose of this project to fill the gap in the current market of games that has no text-based RPG game with seamless integration of MIPS architecture. Development of a new MIPS-based text adventure game will, therefore, bring adventure with a classic charm to the new system of game design.

3. PROPOSED SOLUTION

1.1 3.1. FEATURES OF THE PROJECT

The core mechanics of a simplified RPG experience will be implemented in an RPG game in MIPS Assembly.

- **Character Creation:** Create a character with basic stats (health, attack, and defense) to be created by the player him/herself.
- **Combat System:** In this case there will be one turn-based combat system concerning player fights between random generated monsters. The system will handle damage calculation and health management.
- **Leveling System:** In this case, a player defeats monsters and gains XP before leveling up, which increases health, attack, and defense stats.
- **Random Events:** The game will have random events like encounter with the enemy, finding treasure, and resting to recover health.
- **Item Management:** The player can collect items such as health potions, which he can use during battles to recover health.
- **Boss Battles:** There will be a final battle with a boss, which will be much more powerful and have stats that will be difficult to beat.
- **Score Feature:** The game will include a scoring system that tracks the player's total score, displayed in the inventory details. This feature encourages players to engage with the game mechanics and strive for higher achievements.

1.2 3.2. METHODOLOGY

Development of the RPG game in MIPS Assembly shall be approached in a structured manner, breaking down the game's complexity into smaller modules.

a. Planning Phase:
- Identify key game mechanics and features (combat, character stats, item management).
- Design the structure of the game, including how data (such as character stats and items) will be stored and manipulated in memory.

b. Design Phase:
- Develop pseudocode and flowcharts for combat mechanics, random event generation, and other systems.
- Determine the memory organization and register usage for the game state.

c. Development Phase:
- Implement the main game loop, which will include random events, combat, leveling up, and item usage.
- Implement each module of the game in MIPS Assembly, including the combat system, random event generation, and leveling system.

d. Testing Phase:

- Continuously test each module (for example, combat system, random events) for correctness and efficiency.
- Debugging issues concerning memory management and system calls as MIPS is very sensitive to how registers are used and memory addressed.

e. Deployment Phase:

- Assemble the final code and package the game for demonstration or submission.

1.3 3.3. TECHNOLOGIES TO BE USED

- **Programming Language:** MIPS Assembly Language
- **Development Environment:** MIPS Simulator (Mars, QtSpim)
- **Documentation:** MS Word for in-code documentation, project reports, and user manuals.
- **ASCII Display:** Use to display the game elements on the console, using ASCII characters to reflect the progressing story.

4. PROJECT SCOPE

The project will be text-based RPG game simulation through MIPS Assembly. It will focus on core RPG elements such as character creation, turn-based combat system, random event generation, leveling mechanics, and item management. The game is simulated entirely through assembly code for a low-level understanding of memory management, randomness, and system calls. It will not contain complex graphical representation, multi-user features, and animation as the game would be made within a text-based interface and under the limitations of MIPS Assembly.

5. MODULE DISTRIBUTION

➢ Fiza Khan: Game Logic & Combat System

1.3.1.1 Responsibilities:

- **Game Flow**:
  - Design and implement the main game loop (i.e., character creation, event generation, and exploration).
  - Manage the turn-based combat system (player attacking, monster attacking, using potions).
- **Combat Mechanics**:
  - Develop the logic for calculating damage in combat (using attack and defense stats).
  - Implement different combat actions: **Attack**, **Defend**, and **Use Potion**.
- **Leveling and XP**:
  - Implement experience points (XP) and leveling up system.
  - Add stat boosts (e.g., health, attack) when leveling up.

➢ M Abubakar Afzal: Character Initialization & Inventory System

1.3.1.2 ***Responsibilities:***

- **Character Creation**:
  - Implement character creation (choose name, initialize stats: health, attack, defense).
  - Ensure the player can create a new character at the start of the game.
- **Inventory System**:
  - Create inventory management: **health potions**, **gold**, and other possible items.
  - Implement functionality to **use health potions** during combat to heal the player.
- **Item/Gold Handling**:
  - Handle gold earning (from battles or treasures) and spending (in shops, for example).

➢ Alishba Maqsood: Random Event Generation & Exploration

1.3.1.3 ***Responsibilities:***

- **Random Events**:
  - Create the logic for generating random events during exploration (battle, treasure, safe rest).
  - Implement the random number generator to decide between different types of events.
- **Battle Generation**:
  - Create monster stats (health, attack, defense) randomly for each battle.
  - Implement monster AI to decide when to attack and how to handle different actions (e.g., attacking player, using special abilities).
- **Treasure & Safe Rest**:
  - Generate events where the player can find **treasure** (gold, potions) or **rest** (heal full health).

➢ Mustajab Haider Cheema: Boss Battle & Game End Logic

1.3.1.4 ***Responsibilities:***

- **Boss Battle**:
  - Implement a boss monster with enhanced stats (e.g., more health, unique attacks).
  - Design the special abilities for the boss monster (e.g., healing, dealing massive damage).
  - Create a victory condition when the player defeats the boss.
- **Game End Logic**:
  - Handle the end of the game (win or lose).
  - Display the victory message upon defeating the boss and showing stats (XP, gold earned).
  - Implement any final screens or messages (such as “Game Over” or “You Win!”).
- **Progress Saving**:
  - Optional: Implement a system to save and load the player's progress (e.g., current level, health, inventory).

6. CODE

```
.data
player_name_buffer:      .space 20
player_gender_buffer:    .space 10
player_current_health:   .word 100
player_base_attack:      .word 10
player_base_defense:     .word 5
player_experience:       .word 0
player_current_level:    .word 1
player_potion_count:     .word 2

monster_health: .word 0
encounter_count: .word 0   # Counter for encounters
encounter: .word 0   # Counter for encounters
prompt: .asciiz "\n\nChoose your action (1: Attack, 2: Heal): "
attack_message: .asciiz "\nYou attack the monster!\n"
heal_message: .asciiz "\nYou heal yourself!\n"
win_message: .asciiz "\nWow! You defeated the monster!\n"
lose_message: .asciiz "\nThe monster defeated you!\n"

 # Enemy Data
enemy_current_health:    .word 50
enemy_attack_power:      .word 15
enemy_defense_power:     .word 7

# Messages
title: .asciiz "\n===============WELCOME TO RPG GAME SIMULATION=================="
message_enter_name:      .asciiz "\n\nEnter your name: "
message_choose_gender:   .asciiz "\nEnter your gender (M/F): "
message_choose_class:    .asciiz "\nChoose your class:\n1. Warrior\n2. Mage\n\nEnter choice: "
message_avatar_info:     .asciiz "\nAvatar Information: \nName: "
msg_gender: .asciiz "Gender: "
message_combat_start:    .asciiz "\nCombat begins!\n"
message_player_turn:     .asciiz "\n\nYour turn:\n1. Attack\n2. Defend\n3. Use Potion\nEnter choice: "
message_enemy_turn:      .asciiz "\nEnemy attacks you!\n"
message_random_event:    .asciiz "\nA random event occurred!\n"
message_encounter_enemy: .asciiz "\n\nYou encountered an enemy!\n"
message_found_treasure:  .asciiz "\nCONGRATS! You found a treasure and gained XP!\n"
message_rest_event:      .asciiz "\nFOUND RESTING PACK!! You rested and recovered health!\n"
message_no_potions:      .asciiz "\nYou have no potions left!\n"
message_game_over:       .asciiz "\nGame Over! Thank You For Playing!\n"
message_final_health:    .asciiz "Health: "
message_final_xp:        .asciiz "\tTotal XP Gained: "
message_final_level:     .asciiz "\tLevel: "
message_portions:        .asciiz "\tPortions: "
message_inventory:       .asciiz "\nInventory Details:\n"
message_level_up:        .asciiz "\nLevel Up! You have gained new stats!\n"
message_enemy_attack:    .asciiz "\nThe enemy attacks you!\n"
message_healed:          .asciiz "\nYou have been healed by the potion!\n"  # Added this missing label
max_health_msg: .asciiz "You are at full health!\n"
message_enemy_health:    .asciiz "\nEnemy health: "
message_enemy_defeated: .asciiz "\nCongrats! you have defeated the enemy."
message_player_defeated: .asciiz "\nAhh! U have been defeated :("
levelup: .asciiz "\nLevel up!"
defended: .asciiz"\nYou defended yourself!"
```

```
dash: .asciiz "\n-----------------------------------------------------------------"
newline: .asciiz "\n"
question: .asciiz "\nDo you want to play again? (y/n)\n"
again_buffer:.space 10
yes:        .asciiz "You chose to play again.\n"
no:         .asciiz "\nGoodbye!\n"
invalid:    .asciiz "Invalid input. Please enter 'y' or 'n'.\n"
boss_name:  .asciiz "Dark Overlord\n"
boss_damage: .word 30
boss_health   .word 200  # Base health of the boss
boss_current_health: .word 200  # This will reset between fights

# Messages
boss_message:    .asciiz "\nDark Overlord, The Final Boss appears! Fight for your survival.\n"
attack1: .asciiz "\nThe boss made initial attack!\n"
attack2: .asciiz "\nOh no! The boss attacked you even harder! Your health is shrinking.\n"
attack3: .asciiz "\nThe boss made its final big attack!\n"
round_complete:   .asciiz "\nRound complete! Get ready for the next battle...\n"
final_victory:    .asciiz "\nYou have defeated the Dark Overlord in all three rounds! Victory is yours!\n"
game_over_msg:    .asciiz "\m\nYou were defeated by the Dark Overlord. Game Over.\n"

goblin:  .asciiz "\nGoblin Attcked you!! Save yourself.\n"
Orc:    .asciiz "\n\nOrc Attcked you! Hurry up.\n"
Dragon:  .asciiz "\nDragon Attacked you!.\n"

 monster_types:
       .word 30, 5      # Goblin (Health, Damage)
       .word 50, 10     # Orc (Health, Damage)
       .word 70, 15     # Dragon (Health, Damage)

.text
.globl main

main:

   li $v0, 4
   la $a0, title
   syscall

   # Character Creation - Name and Gender
   li $v0, 4
   la $a0, message_enter_name
   syscall

   li $v0, 8                # Read string (name)
   la $a0, player_name_buffer
   li $a1, 20               # Max length of name
   syscall

   li $v0, 4
   la $a0, message_choose_gender
   syscall

   li $v0, 8                # Read string (gender)
   la $a0, player_gender_buffer
   li $a1, 10               # Max length of gender input
   syscall
```

```
    # Display Avatar Info
    li $v0, 4
    la $a0, message_avatar_info
    syscall

    # Display name
    li $v0, 4
    la $a0, player_name_buffer
    syscall

 # Display Avatar Info
    li $v0, 4
    la $a0,msg_gender
    syscall

    # Display gender
    li $v0, 4
    la $a0, player_gender_buffer
    syscall


    li $t1, 0            # Encounter count
    sw $t1, encounter_count

class:
    li $v0, 4
    la $a0, dash
    syscall

    # Choose Class
    li $v0, 4
    la $a0, message_choose_class
    syscall

    li $v0, 5                # Read choice for class
    syscall
    move $t0, $v0  # Store class choice
    li $t1, 1
    beq $t0, $t1, setup_warrior
    li $t1, 2
    beq $t0, $t1, setup_mage

setup_warrior:
    li $t0, 100# Higher health for warrior
    sw $t0, player_current_health
    li $t0, 15    # Higher attack for warrior
    sw $t0, player_base_attack
    li $t0, 10    # Higher defense for warrior
    sw $t0, player_base_defense
    j initialize_game

setup_mage:
    li $t0, 50   # Lower health for mage
    sw $t0, player_current_health
    li $t0, 20    # Higher attack for mage
    sw $t0, player_base_attack
```

```
    li $t0, 5     # Lower defense for mage
    sw $t0, player_base_defense
    j initialize_game

initialize_game:
   li $v0, 4
    la $a0, dash
    syscall

    li $v0, 4
    la $a0, message_combat_start
    syscall

next_encounter:

    lw $t1, encounter_count   # Load encounter count

    # Determine which monster to show based on encounter count
    beq $t1, 0, print_goblin   # If count is 0, print Goblin
    beq $t1, 1, find_treasure
    beq $t1, 2, enemy_encounter
    beq $t1, 3, print_orc       # If count is 1, print Orc
       beq $t1, 4, rest_and_recover       # If count is 1, print Orc
    beq $t1, 5, print_dragon    # If count is 2, print Dragon
    beq $t1, 6, enemy_encounter
    beq $t1, 7, find_treasure
     beq $t1, 8, enemy_encounter
        beq $t1, 9, find_treasure
         beq $t1, 10, next_battle


end_game:
    li $v0, 10                  # Exit syscall
    syscall                     # Exit program

print_goblin:
   li $v0, 4
    la $a0, dash
    syscall

    la $a0, goblin      # Load address of monster names
    li $v0, 4                   # Print string syscall
    syscall                     # Print Goblin name

    li $v0, 4
    la $a0, dash
    syscall

    la $t2, monster_types       # Load base address of monster_types
    lw $t3, ($t2)               # Load Goblin health into t3
    sw $t3, monster_health       # Store Goblin health

  # Generate a random damage amount for the enemy attack (e.g., 5 to 15 damage)
 li $t0, 10                  # Minimum damage

    # Load the player's current health
    lw $t3, player_current_health
```

```
    # Calculate the player's new health after taking damage
    sub $t3, $t3, $t0          # Subtract damage from player's health

    # Ensure player's health does not go below 0
    bltz $t3, zero_health_limit

    # Update the player's health
    sw $t3, player_current_health
 jal show_player_inventory2
    j player_action             # Jump to player action selection

print_orc:
   li $v0, 4
    la $a0, dash
    syscall

   la $a0, Orc      # Load address of monster names
    li $v0, 4                  # Print string syscall
    syscall                     # Print Goblin name

    li $v0, 4
    la $a0, dash
    syscall

    la $t2, monster_types       # Load base address of monster_types
    addi $t2, $t2, 8            # Move to Orc's data (second entry)
    lw $t3, ($t2)               # Load Orc health into t3
    sw $t3, monster_health       # Store Orc health

  # Generate a random damage amount for the enemy attack (e.g., 5 to 15 damage)
 li $t0, 15                  # Minimum damage

    # Load the player's current health
    lw $t3, player_current_health

    # Calculate the player's new health after taking damage
    sub $t3, $t3, $t0          # Subtract damage from player's health

    # Ensure player's health does not go below 0
    bltz $t3, zero_health_limit

    # Update the player's health
    sw $t3, player_current_health
 jal show_player_inventory2
    j player_action             # Jump to player action selection

print_dragon:
    li $v0, 4
    la $a0, dash
    syscall

    la $a0, Dragon      # Load address of monster names
    li $v0, 4                  # Print string syscall
    syscall                     # Print Dragon name

    li $v0, 4
```

```
    la $a0, dash
    syscall

    la $t2, monster_types       # Load base address of monster_types
    addi $t2, $t2, 16           # Move to Dragon's data (third entry)
    lw $t3, ($t2)               # Load Dragon health into t3
    sw $t3, monster_health      # Store Dragon health

  # Generate a random damage amount for the enemy attack (e.g., 5 to 15 damage)
 li $t0, 20                 # Minimum damage

    # Load the player's current health
    lw $t3, player_current_health

    # Calculate the player's new health after taking damage
    sub $t3, $t3, $t0          # Subtract damage from player's health

    # Ensure player's health does not go below 0
    bltz $t3, zero_health_limit

    # Update the player's health
    sw $t3, player_current_health

     jal show_player_inventory2

player_action:

   li $v0, 4                  # Print prompt message syscall
   la $a0, prompt             # Load prompt message address
   syscall                    # Print prompt message

   li $v0, 5                  # Read integer syscall for action choice (1 or 2)
   syscall
   move $t4, $v0              # Store choice in t4

   beq $t4, 1, attack         # If choice is Attack (1), jump to attack logic
   beq $t4, 2, heal           # If choice is Heal (2), jump to heal logic

attack:
   li $v0, 4                  # Print attack message syscall
   la $a0, attack_message
   syscall

   lw $t5, monster_health       # Load current monster health into t5
   la $t6, monster_types
   lw $t7, ($t6)              # Load damage from Goblin/Orc/Dragon into t6

   subu $t5,$t5,$t7
   sw $t5 ,monster_health

   bgtz $t5 , continue

   li $v0 ,4
   la $a0 ,win_message
   syscall

     lw $t4, player_current_health    # Load the enemy's current health
```

```
    blez $t4, player_defeated       # If enemy health is less than or equal to 0, jump to enemy_defeated

   j continue

heal:
   li $v0 ,4
   la $a0 ,heal_message
   syscall

   lw $t8 ,player_current_health
   addi $t8,$t8 ,20
   sw $t8 ,player_current_health

    lw $t4, player_current_health    # Load the enemy's current health
    blez $t4, player_defeated       # If enemy health is less than or equal to 0, jump to enemy_defeated

continue:

    lw $t1, encounter_count      # Load current encounter count into $t1
    addi $t1, $t1, 1             # Increment the encounter count by 1
    sw $t1, encounter_count       # Store the updated count back to memory

   j next_encounter

   li $v0 ,4
   la $a0 ,lose_message

enemy_encounter:
   li $v0, 4
    la $a0, dash
    syscall

 # Display player's turn
    li $v0, 4
    la $a0,message_encounter_enemy
    syscall

game_loop:
lw $t4, enemy_current_health   # Load the enemy's current health
    blez $t4, enemy_defeated       # If enemy health is less than or equal to 0, jump to enemy_defeated

    lw $t4, player_current_health    # Load the enemy's current health
    blez $t4, player_defeated       # If enemy health is less than or equal to 0, jump to enemy_defeated


    # Show player's health and inventory
    jal show_player_inventory

    # Display player's turn
    li $v0, 4
    la $a0, message_player_turn
    syscall

    # Wait for player action
    li $v0, 5
    syscall
    move $t1, $v0  # Store the player's choice
```

```
    # Handle player's choice for action
    li $t2, 1
    beq $t1, $t2, action_player_attack  # If choice is 1, attack
    li $t2, 2
    beq $t1, $t2, action_player_defend  # If choice is 2, defend
    li $t2, 3
    beq $t1, $t2, action_use_potion     # If choice is 3, use potion

    # If the choice is invalid, restart the loop
    j game_loop

action_player_attack:
    # Attack logic (damage calculation)
    lw $t0, player_base_attack
    lw $t1, enemy_current_health
    sub $t1, $t1, $t0
    sw $t1, enemy_current_health

    # Jump to enemy's turn after attack
    j action_enemy_turn

action_player_defend:
    # Defense logic (damage reduction)
    li $t2, 2            # Defense multiplier (reduce damage by half)
    lw $t0, enemy_attack_power   # Load enemy's attack power
    div $t0, $t2         # Divide enemy attack by defense multiplier
    mflo $t0             # Get result (reduced damage)

    lw $t1, player_current_health   # Load player's current health
    sub $t1, $t1, $t0      # Reduce player's health by the defense-modified damage
    sw $t1, player_current_health   # Store new health back

 li $v0, 4
    la $a0, defended
    syscall                     # Print the healing message


    j continue  # Return to the main game loop

action_use_potion:
    # Check if the player has any potions left
    lw $t1, player_potion_count    # Load the player's potion count
    beqz $t1, no_potions_left      # If potion count is 0, skip the potion use

    li $t2, 20                  # Set the healing amount (20 health points)
    lw $t1, player_current_health  # Load current health of the player
    add $t1, $t1, $t2             # Add healing amount to player's current health

    # Update player health after using potion
    sw $t1, player_current_health  # Store new health back to player

    # Decrease potion count after use
    lw $t1, player_potion_count    # Load current potion count
    sub $t1, $t1, 1               # Decrease potion count by 1
    sw $t1, player_potion_count    # Store the updated potion count
```

```
    # Message indicating potion use
    li $v0, 4
    la $a0, message_healed
    syscall                    # Print the healing message

    j continue  # Return to the main game loop

no_potions_left:
    li $v0, 4
    la $a0, message_no_potions    # Message: "You have no potions left!"
    syscall
    j game_loop


show_player_inventory:
    li $v0, 4
    la $a0, dash
    syscall

    # Display player's potion count
    li $v0, 4
    la $a0, message_inventory       # Message: "Inventory Details: "
    syscall

    # Display player's current health
    li $v0, 4
    la $a0, message_final_health    # Message: "Remaining Health: "
    syscall

    li $v0, 1
    lw $a0, player_current_health
    syscall

    # Display player's current level
    li $v0, 4
    la $a0, message_final_level     # Message: "Final Level: "
    syscall

    li $v0, 1
    lw $a0, player_current_level
    syscall

    # Display player's total experience
    li $v0, 4
    la $a0, message_final_xp        # Message: "Total XP Gained: "
    syscall

    li $v0, 1
    lw $a0, player_experience
    syscall

    li $v0, 4
    la $a0, message_portions        # Message: "Portions: "
    syscall

    li $v0, 1
    lw $a0, player_potion_count
```

```
    syscall

     # Display enemy current health
    li $v0, 4
    la $a0, message_enemy_health  # Message: "Enemy Health: "
    syscall

    li $v0, 1
    lw $a0, enemy_current_health
    syscall


    jr $ra  # Return to the main game loop

show_player_inventory2:
    # Display player's potion count
    li $v0, 4
    la $a0, message_inventory       # Message: "Inventory Details: "
    syscall

    # Display player's current health
    li $v0, 4
    la $a0, message_final_health    # Message: "Remaining Health: "
    syscall

    li $v0, 1
    lw $a0, player_current_health
    syscall

    # Display player's current level
    li $v0, 4
    la $a0, message_final_level     # Message: "Final Level: "
    syscall

    li $v0, 1
    lw $a0, player_current_level
    syscall

    # Display player's total experience
    li $v0, 4
    la $a0, message_final_xp        # Message: "Total XP Gained: "
    syscall

    li $v0, 1
    lw $a0, player_experience
    syscall

    li $v0, 4
    la $a0, message_portions        # Message: "Portions: "
    syscall

    li $v0, 1
    lw $a0, player_potion_count
    syscall

    jr $ra  # Return to the main game loop
```

```
action_enemy_turn:

    # Generate a random damage amount for the enemy attack (e.g., 5 to 15 damage)
 li $t0, 5                # Minimum damage
li $t1, 30                # Maximum damage (increased)
    jal generate_random        # Call the function to generate random damage

    # Store the damage amount in $t2 (random damage between 5 and 15)
    move $t2, $v0

    # Load the player's current health
    lw $t3, player_current_health

    # Calculate the player's new health after taking damage
    sub $t3, $t3, $t2          # Subtract damage from player's health

    # Ensure player's health does not go below 0
    bltz $t3, zero_health_limit

    # Update the player's health
    sw $t3, player_current_health

    # Display message that enemy attacked
    li $v0, 4
    la $a0, message_enemy_attack
    syscall

    j continue  # Return to the main game loop

enemy_defeated:
li $v0, 4
    la $a0, dash
    syscall

   # Display message that enemy attacked
    li $v0, 4
    la $a0, message_enemy_defeated
    syscall
    j play_again

player_defeated:

    j zero_health_limit

zero_health_limit:
li $v0, 4
    la $a0, dash
    syscall

   li $v0, 4
    la $a0, message_player_defeated
    syscall

    li $t3, 0
    sw $t3, player_current_health   # Set health to 0 (dead)

    # Display player's experience
```

```
    li $v0, 4
    la $a0, message_final_xp   # "Experience: "
    syscall

    li $v0, 1
    lw $a0, player_experience
    syscall

    # Display player's level
    li $v0, 4
    la $a0, message_final_level   # "Level: "
    syscall

    li $v0, 1
    lw $a0, player_current_level
    syscall

    j game_over

game_over:
    li $v0, 4
    la $a0, message_game_over
    syscall

    j play_again

# Function to generate a random number between two values (inclusive)
generate_random:
    li $v0, 42            # Random number syscall
    syscall
    div $v0, $t1           # Divide by the max value (upper bound)
    mfhi $v0               # Get the remainder
    add $v0, $v0, $t0      # Add the minimum value (lower bound)
    jr $ra

find_treasure:
   li $v0, 4
    la $a0, dash
    syscall

lw $t1, player_current_level
    addi $t1, $t1, 1  # Add 10 XP as an example for finding treasure
    sw $t1, player_current_level

      li $v0, 4
    la $a0, levelup
    syscall

    li $v0, 4
    la $a0, message_found_treasure
    syscall

    # Award XP or items (adjust the player's experience as needed)
    lw $t1, player_experience
    addi $t1, $t1, 10  # Add 10 XP as an example for finding treasure
    sw $t1, player_experience
```

```
    # Optionally, display updated XP or other benefits here
    j continue

rest_and_recover:
   li $v0, 4
    la $a0, dash
    syscall

 lw $t4, player_current_health    # Load the enemy's current health
    blez $t4, player_defeated       # If enemy health is less than or equal to 0, jump to enemy_defeated

    li $v0, 4
    la $a0, message_rest_event
    syscall

    # Recover health by 20 points
    lw $t1, player_current_health
    addi $t1, $t1, 20  # Heal 20 health points

    # Ensure health doesn't exceed max health
    li $t2, 100  # Max health
    blt $t1, $t2, continue_rest
    move $t1, $t2  # Set health to max if it exceeds

continue_rest:
 lw $t4, player_current_health    # Load the enemy's current health
    blez $t4, player_defeated       # If enemy health is less than or equal to 0, jump to enemy_defeated

    sw $t1, player_current_health

    j continue

next_battle:

    lw $t1, encounter   # Load encounter count

    # Determine which monster to show based on encounter count
    beq $t1, 0, first   # If count is 0, print Goblin
    beq $t1, 1, second
    beq $t1, 2, third

first:
   li $v0, 4
    la $a0, dash
    syscall

    # Display boss message
    li $v0, 4
    la $a0, boss_message
    syscall

    li $v0, 4
    la $a0, dash
    syscall

    # Display boss message
    li $v0, 4
```

```
   la $a0, attack1
   syscall

     li $v0, 4
   la $a0, dash
   syscall

 # Generate a random damage amount for the enemy attack
li $t0, 20                # Minimum damage

   # Load the player's current health
   lw $t3, player_current_health

   # Calculate the player's new health after taking damage
   sub $t3, $t3, $t0         # Subtract damage from player's health

   # Ensure player's health does not go below 0
   bltz $t3, zero_health_limit

   # Update the player's health
   sw $t3, player_current_health
jal show_player_inventory2
   j player_action_boss              # Jump to player action selection

second:
  li $v0, 4
   la $a0, dash
   syscall

     # Display boss message
   li $v0, 4
   la $a0, round_complete
   syscall
   li $v0, 4
   la $a0, dash
   syscall

    # Display boss message
   li $v0, 4
   la $a0, attack2
   syscall

     li $v0, 4
   la $a0, dash
   syscall

 # Generate a random damage amount for the enemy attack (e.g., 5 to 15 damage)
li $t0, 20               # Minimum damage

   # Load the player's current health
   lw $t3, player_current_health

   # Calculate the player's new health after taking damage
   sub $t3, $t3, $t0         # Subtract damage from player's health

   # Ensure player's health does not go below 0
   bltz $t3, zero_health_limit
```

```
   # Update the player's health
   sw $t3, player_current_health
 jal show_player_inventory2
   j player_action_boss            # Jump to player action selection

third:
   li $v0, 4
   la $a0, dash
   syscall

     # Display boss message
   li $v0, 4
   la $a0, round_complete
   syscall
   li $v0, 4
   la $a0, dash
   syscall

    # Display boss message
   li $v0, 4
   la $a0, attack3
   syscall
      li $v0, 4
   la $a0, dash
   syscall

  # Generate a random damage amount for the enemy attack (e.g., 5 to 15 damage)
 li $t0, 30              # Minimum damage

   # Load the player's current health
   lw $t3, player_current_health

   # Calculate the player's new health after taking damage
   sub $t3, $t3, $t0         # Subtract damage from player's health

   # Ensure player's health does not go below 0
   bltz $t3, zero_health_limit

   # Update the player's health
   sw $t3, player_current_health

    jal show_player_inventory2
 j player_action_boss

player_action_boss:

  li $v0, 4               # Print prompt message syscall
  la $a0, prompt             # Load prompt message address
  syscall                  # Print prompt message

  li $v0, 5               # Read integer syscall for action choice (1 or 2)
  syscall
  move $t4, $v0             # Store choice in t4

  beq $t4, 1, attackboss         # If choice is Attack (1), jump to attack logic
  beq $t4, 2, healboss           # If choice is Heal (2), jump to heal logic
```

```
attackboss:
   li $v0, 4                # Print attack message syscall
   la $a0, attack_message
   syscall

   lw $t5, boss_health       # Load current monster health into t5
   lw $t6, boss_damage

   subu $t5,$t5,$t7
   sw $t5 ,boss_health

   bgtz $t5 , continue2


   blez $t4, victory      # If enemy health is less than or equal to 0, jump to enemy_defeated

     lw $t4, player_current_health
    blez $t4, defeat       # If enemy health is less than or equal to 0, jump to enemy_defeated

     li $v0, 4
    la $a0, dash
    syscall

lw $t1, player_current_level
    addi $t1, $t1, 1  # Add 10 XP as an example for finding treasure
    sw $t1, player_current_level

      li $v0, 4
    la $a0, levelup
    syscall

    li $v0, 4
    la $a0, message_found_treasure
    syscall

    # Award XP or items (adjust the player's experience as needed)
    lw $t1, player_experience
    addi $t1, $t1, 10  # Add 10 XP as an example for finding treasure
    sw $t1, player_experience

   j continue2

healboss:
   li $v0 ,4
   la $a0 ,heal_message
   syscall

   lw $t8 ,player_current_health
   addi $t8,$t8 ,50
   sw $t8 ,player_current_health

     lw $t4, player_current_health    # Load the enemy's current health
    blez $t4, defeat       # If enemy health is less than or equal to 0, jump to enemy_defeated

j continue2
```

```
defeat:
    lw $t4, player_current_health    # Load the enemy's current health

   blez $t4, final_game_over    # Check if player is defeated

   # Continue the battle loop
    jal show_player_inventory2

   j play_again
victory:
  li $v0, 4
   la $a0, dash
   syscall

   # Player wins all rounds
   li $v0, 4
   la $a0, final_victory
   syscall

      li $v0, 4
   la $a0,  message_game_over
   syscall


   jal show_player_inventory2

   j play_again

final_game_over:
   # Display game over message
   li $v0, 4
   la $a0, game_over_msg
   syscall

  exit_game:
  li $v0, 4
   la $a0, no
   syscall

  li $v0,10
  syscall

continue2:

   lw $t1, encounter    # Load current encounter count into $t1
   addi $t1, $t1, 1            # Increment the encounter count by 1
   sw $t1, encounter    # Store the updated count back to memory

   lw $t1, encounter    # Load current encounter count into $t1
   li $t2,3
   beq $t2,$t1,check

  j next_battle

  li $v0 ,4
  la $a0 ,lose_message
```

```
check:
lw $t8 ,player_current_health
bgtz $t8,victory
j defeat

play_again:
   li $v0, 4
   la $a0, dash
   syscall

   li $v0, 4                    # Print string syscall
   la $a0, question                # Load the question prompt
   syscall

   li $v0, 8                    # Read string syscall
   la $a0,  again_buffer                # Load buffer for input
   li $a1, 4                    # Max input size
   syscall

   lb $t0, again_buffer                # Load the first character of the response
   beq $t0, 'y', main          # If 'y', go to play_again
   beq $t0, 'n', exit_game          # If 'n', go to exit_program

invalid_input:
   li $v0, 4                    # Print string syscall
   la $a0, invalid                # Load the invalid input message
   syscall
   j play_again
```

7. Interfaces

```
===============WELCOME TO RPG GAME SIMULATION==================

Enter your name: Alishba

Enter your gender (M/F): F

Avatar Information:
Name: Alishba
Gender: F

-------------------------------------------------------------------
Choose your class:
1. Warrior
2. Mage

Enter choice: 1

-------------------------------------------------------------------
Combat begins!

-------------------------------------------------------------------
Goblin Attcked you!! Save yourself.

-------------------------------------------------------------------
Inventory Details:
Health: 90      Level: 1        Total XP Gained: 0      Portions: 2

Choose your action (1: Attack, 2: Heal): 2

You heal yourself!

-------------------------------------------------------------------
Level up!
CONGRATS! You found a treasure and gained XP!

-------------------------------------------------------------------

You encountered an enemy!

-------------------------------------------------------------------
Inventory Details:
Health: 110     Level: 2        Total XP Gained: 10     Portions: 2
Enemy health: 50
```

```
Your turn:
1. Attack
2. Defend
3. Use Potion
Enter choice: 2

You defended yourself!
-----------------------------------------------------------------

Orc Attcked you! Hurry up.

-----------------------------------------------------------------
Inventory Details:
Health: 88      Level: 2        Total XP Gained: 10     Portions: 2

Choose your action (1: Attack, 2: Heal): 1

You attack the monster!

-----------------------------------------------------------------
FOUND RESTING PACK!! You rested and recovered health!

-----------------------------------------------------------------
Dragon Attacked you!.

-----------------------------------------------------------------
Inventory Details:
Health: 80      Level: 2        Total XP Gained: 10     Portions: 2

Choose your action (1: Attack, 2: Heal): 2

You heal yourself!

-----------------------------------------------------------------

You encountered an enemy!

-----------------------------------------------------------------
Inventory Details:
Health: 100     Level: 2        Total XP Gained: 10     Portions: 2
Enemy health: 50
```

```
Your turn:
1. Attack
2. Defend
3. Use Potion
Enter choice: 3

You have been healed by the potion!

-----------------------------------------------------------------
Level up!
CONGRATS! You found a treasure and gained XP!

-----------------------------------------------------------------

You encountered an enemy!

-----------------------------------------------------------------
Inventory Details:
Health: 120     Level: 3        Total XP Gained: 20     Portions: 1
Enemy health: 50

Your turn:
1. Attack
2. Defend
3. Use Potion
Enter choice: 1

The enemy attacks you!

-----------------------------------------------------------------
Level up!
CONGRATS! You found a treasure and gained XP!

-----------------------------------------------------------------
Dark Overlord, The Final Boss appears! Fight for your survival.

-----------------------------------------------------------------
The boss made initial attack!

-----------------------------------------------------------------
Inventory Details:
Health: 83      Level: 4        Total XP Gained: 30     Portions: 1
```

```
-----------------------------------------------------------------
Round complete! Get ready for the next battle...

-----------------------------------------------------------------
Oh no! The boss attacked you even harder! Your health is shrinking.

-----------------------------------------------------------------
Inventory Details:
Health: 113     Level: 4        Total XP Gained: 30     Portions: 1

Choose your action (1: Attack, 2: Heal): 1

You attack the monster!

-----------------------------------------------------------------
Round complete! Get ready for the next battle...

-----------------------------------------------------------------
The boss made its final big attack!

-----------------------------------------------------------------
Inventory Details:
Health: 83      Level: 4        Total XP Gained: 30     Portions: 1

Choose your action (1: Attack, 2: Heal): 1

You attack the monster!

-----------------------------------------------------------------
You have defeated the Dark Overlord in all three rounds! Victory is yours!

Game Over! Thank You For Playing!

Inventory Details:
Health: 83      Level: 4        Total XP Gained: 30     Portions: 1
-----------------------------------------------------------------
Do you want to play again? (y/n)
n

Goodbye!

-- program is finished running --
```

8. CONCLUSION

In conclusion , the RPG game simulation developed in MIPS Assembly successfully integrates essential RPG mechanics, providing an engaging text-based experience. The project met its objectives by implementing character creation, combat systems, and random events while overcoming challenges related to memory management and debugging. Future enhancements could explore graphical elements or multiplayer features to expand gameplay. This project not only highlights the potential of assembly language in game design but also serves as a valuable educational resource for understanding low-level programming concepts.

9. REFERENCES

- M. Fienup, "MIPS Assembly Language Guide," University of Northern Iowa, 2008. [Online]. Available: http://www.cs.uni.edu/~fienup/cs041s08/lectures/lec20_MIPS.pdf.
- "C# Text-Based RPG Combat System," Stack Overflow, 2017. [Online]. Available: https://stackoverflow.com/questions/43248663/c-sharp-text-based-rpg-combat-system-currency-randomisation.
- "MIPS Assembly Language Programming," YouTube, 2021. [Online]. Available: https://www.youtube.com/watch?v=t7-o-Neoc9E.
- https://github.com/TanByv/MEFQuest
- https://reintech.io/blog/how-to-create-a-text-based-adventure-game-with-python

- "MIPS Assembly Language Programming," Google Books, 2003. [Online]. Available:https://books.google.com/books/about/MIPS_Assembly_Language_Programming.html?id=d9QPBQAAQBAJ.